



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Introduction to Python for Data Science

DSECLPFDS / AIMLCPFDS

- Variables
- Types
- Arithmetic operators
- Boolean logic
- Strings
- Printing

Agenda for CS#2



- Immutable & Mutable Data Structures
- Lists
- Tuples
- Dictionaries
- Sets
- Conditional Statements
- Exception Handling

Objects, Value, Identity, and Type

In Python, everything is an object. Every Python object has three core characteristics that define it at a foundational level. These characteristics are:

1. Value : This represents the actual data or content stored within the object
2. Type : The type of an object determines which class that object derives from
3. Identity: This identity is a unique identifier that distinguishes one object from others. An object's identity is a read-only property

Immutable & Mutable Objects



Mutable Objects:

Some objects are mutable, meaning their values can change throughout the execution of a program. In practice, this means they maintain the same id even after a change in value. This is known as an ‘in-place’ change because it doesn’t require the creation of a new object to change in value. Mutable objects include lists, dictionaries, and sets.

Immutable objects:

Immutable objects on the other hand can not change in value during the execution of a program. The only way to alter the value of a variable that points to an immutable object, is to create an entirely new object containing the altered value. In other words, immutable objects can not be changed ‘in-place.’ Numbers, String, tuples are examples.

- Effectively lists that are immutable (i.e. can't be changed)

```
fruits = ("apple", "orange", "tomato", "banana") # now the tomato is a fruit forever
print(type(fruits))
print(fruits)
```

```
<class 'tuple'>
('apple', 'orange', 'tomato', 'banana')
```

```
fruits = ["apple", "orange", "tomato", "banana"] # a list of strings
print(type(fruits))
print(fruits)
```

```
<class 'list'>
['apple', 'orange', 'tomato', 'banana']
```

Lists with integers



range() - a function that generates a sequence of numbers as a list

```
nums = list(range(10))  
print(nums)
```

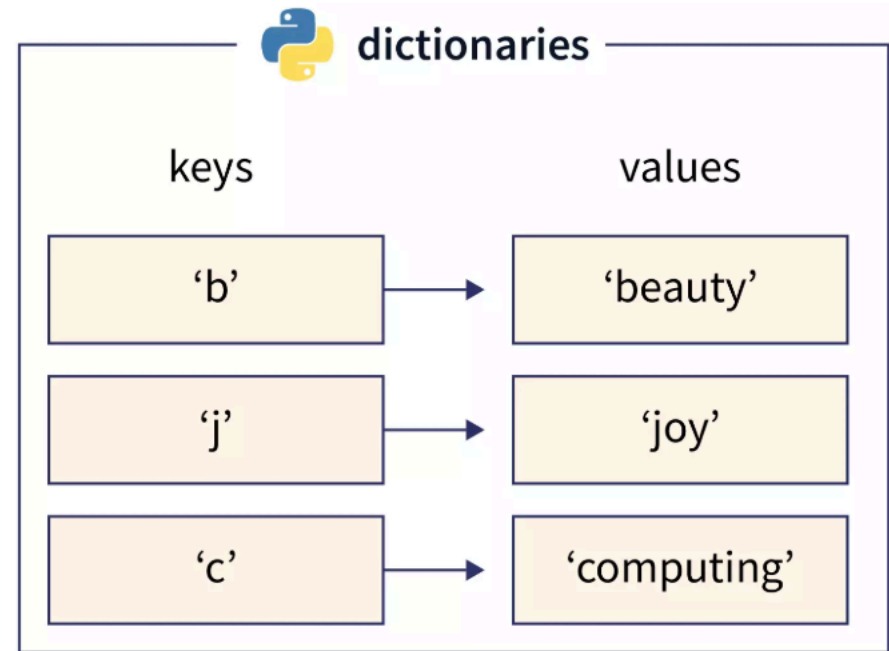
```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
nums = list(range(0, 100, 5))  
print(nums)
```

```
[0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85,  
90, 95]
```


- Slicing – obtain a particular set of sub-elements from a data structure.
- Very useful and flexible.
- Lists helpful functions
 - max
 - min
 - len

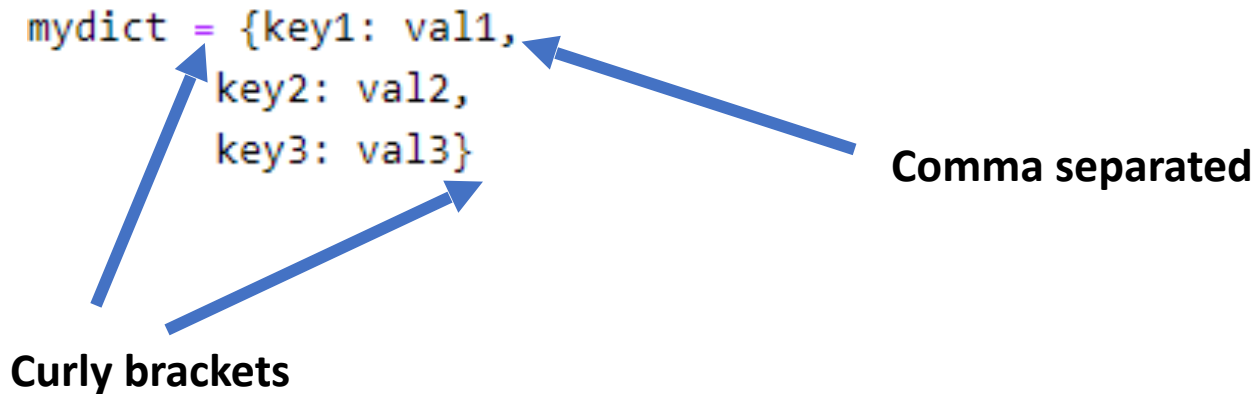
- Similar to actual dictionaries
- They are effectively 2 lists combined – keys and values
- We use the keys to access the values instead of indexing them like a list
- Each value is mapped to a unique key



Dictionary definition



Defined as comma separated **key : value** pairs:



- Values are mapped to a key
- Values are accessed by their key
- Key are unique and are immutable
- Values cannot exist without a key

- Effectively lists that can't contain duplicate items
- Similar functionality to lists
- Can't be indexed or sliced
- Can be created with `{}` or you can convert a list to a set

Data Type	Built-in Class	Mutable
Numbers	int, float, complex	✗
Strings	str	✗
Tuples	tuple	✗
Bytes	bytes	✗
Booleans	bool	✗
Frozen sets	frozenset	✗
Lists	list	✓
Dictionaries	dict	✓
Sets	set	✓

Conditional Statements

- If Else

```
if answer:
    close program
else:
    continue running program
```

← Conditional statement

← Executed if answer is **True**

← Executed if answer is **False**

Indentation Matters!



- Code is grouped by its indentation
- Indentation is the number of whitespace or tab characters before the code.
- If you put code in the wrong block then you will get unexpected behaviour

Extending if-else blocks



- We can add infinitely more if statements using **elif**
- **elif** = **else** + **if** which means that the previous statements must be false for the current one to evaluate to true

```
if condition1:  
    condition 1 was True  
elif condition2:  
    condition 2 was True  
else:  
    neither condition 1 or condition 2 were True
```

```
for item in itemList:  
    do something to item
```

- Allows us to iterate over a set amount of variables within a data structure. During that we can manipulate each item however we want
- Again, indentation is important here!

- While loop
 - Another useful loop. Similar to the for loop.
 - A while loop doesn't run for a predefined number of iterations, like a for loop. Instead, it stops as soon as a given condition becomes true/false.

- Even if a statement or expression is syntactically correct, it may cause an error when an attempt is made to execute it. Errors detected during execution are called *exceptions* and are not unconditionally fatal